

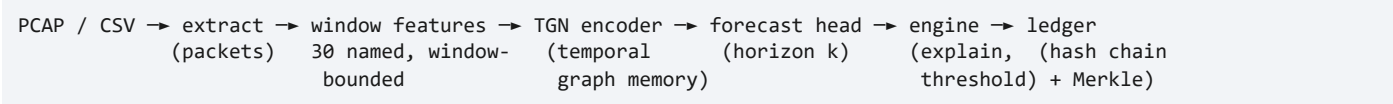
Architecture — Network Attack Forecasting (SIH26153)

Two pages. Numbers are reproducible via `python scripts/make_ablation_table.py`.

1. Problem and unit of prediction

Given network traffic up to time t , forecast the **infiltration probability and ATT&CK stage at $t+k$** , explain each forecast in named features, and commit it to a tamper-evident ledger — fully offline. The unit is **(source host, 15 s window)** on a 5 s stride, never per-flow and never whole-network; the network-level score is the **max** over host scores in a window. Success is measured by **lead time** (seconds between the first alert on the attacking host and the annotated attack completion) at a fixed false-positive budget — not by F1.

2. Pipeline



Ingest & features. A single streaming extractor (`data/packet_features.py`) parses pcap bytes with raw struct offsets (~110k packets/s, bounded memory on a 4.3 GB flood capture) and emits both flow records and per-`(src, window)` packet statistics: TTL mean/variance, TCP window, fragment flags, an 8-bin payload histogram, distinct destination ports/IPs, port entropy, a **sequential-vs-randomised scan ratio**, and retransmission counts (tshark-verified, scapy fallback). `data/windows.py` joins these into a **30-feature, strictly window-bounded** matrix — every value summarises only packets whose own timestamp lies in the window. That constraint is load-bearing: attributing whole-flow totals to a flow's start window leaked forecast-horizon traffic into the present and would have fabricated the headline metric (`docs/decisions/003`).

Identity. Nodes are keyed-HMAC pseudonyms with a per-epoch permutation; node features are role-only (internal/external, hashed /24). No raw IP reaches the model or the ledger.

Encoder. `models/tgn.py` maintains a TGN memory over the host graph; edges are flow events timestamped at **flow end** (a flow's statistics are only known once it completes — the same causality rule as the features). Each host's memory vector at a window boundary is its embedding. Memory is reset between splits so training state never reaches test.

Sequence model. `models/graft.py` is a 2-layer causal Transformer over per-host window sequences — always masked with both a causal mask and a padding mask, verified *bit-identically* by `test_no_future_leakage` (perturbing windows $t+1...T$ leaves the prediction at t unchanged). It carries a Dirichlet evidential head (uncertainty = K/S) and a benign-only reconstruction head as an OOD signal.

Forecast head. Labels are shifted by k , so the model predicts "*attack on this host in k windows*". This is the shipped M7 deliverable; the planned RSSM rollout failed its gate and is recorded as a negative result (`docs/decisions/004`, `docs/limitations.md §4`).

3. Engine, explainability, ledger

`engine/predict.py` runs fully offline from persisted artefacts: it builds features from the input file, scores each host-window against a threshold **fitted from an FPR budget on validation** (never a literal), and for each alert emits a JSON-schema-validated object. The deployed scorer is the cascade's **fast tier** (XGBoost — CPU-cheap and natively TreeSHAP-explainable); the fused TGN+XGB headline in §4 is the eval-side model, and engine-side fusion is roadmap. Each alert object carries: probability, stage, MITRE technique, top-5 **named** features, the top-3 contributing windows (where the attack was forming), and the flagged flows in that window. Attributions are TreeSHAP over the deployed model in named-feature space — the problem statement rules out black-box output, so explanations name `payload_hist_0` or `distinct_dst_ips`, never an embedding index. `engine/technique_map.yaml` maps stage + observed pattern to techniques actually evidenced in our data (internal scan → **T1046**, credential guessing → **T1110**, C2 → **T1071**, availability attack → **T1498**, bulk egress → **T1048**).

Every forecast is appended to an **append-only hash-chained JSONL** with per-batch Merkle roots (`ledger/`). Editing one record breaks that record's hash and the verifier names its exact index; a fully rewritten, self-consistent chain still fails because its head no longer matches the **anchored checkpoint**. Hosts are HMAC-pseudonymised in the chain, and the engine **refuses to write records if the model weights' SHA-256 does not match** the recorded digest, so a ledger entry can never claim provenance it does not have. `ledger/verify_cli.py` lets a judge verify with no network.

4. Evaluation and results

Splits are **day-wise** (train 14-02+16-02, val 28-02, test 02-03); the scaler is fit on train only and the threshold on validation only. With four attack days this makes the splits attack-family-disjoint, so the headline is a cross-family generalisation test. Undetected episodes count as **0 s** lead and are never dropped.

Test split (bot day), 1 % FPR budget:

model	AUROC	F1	median lead	episodes
Fused (rank-mean TGN+XGB) — shipped	0.942	0.382	5008 s	2/2
GRAFT + clamped Time2Vec (best single; val-gated)	0.937	0.480	4170 s	2/2
TGN encoder	0.877	0.035	3595 s	2/2
XGBoost	0.872	0.140	4208 s	2/2
LSTM	0.764	0.015	4202 s	2/2
Logistic regression (graded baseline)	0.573	0.001	0 s	0/2

All rows use one standardised anonymisation key (earlier pre-standardisation numbers were not reproducible and were retired). The headline is the parameter-free **fusion** — TGN and XGBoost make nearly uncorrelated errors (Spearman $\rho \approx 0.11$), so averaging their score *ranks* beats both, and it is selected on validation, never on test.

Forecasting ahead (shipped M7): AUROC **0.913 at k=4 (20 s ahead)** with **2/2 episodes and ~80 min lead**; ranking stays ≈ 0.91 at k=8 (40 s ahead), where the fixed 1 % operating point stops firing (0/2) — disclosed, not hidden. A linear model cannot transfer across attack families at all.

5. Offline guarantee

No cloud APIs, no runtime downloads, no telemetry (Streamlit's usage reporting is explicitly disabled). Dependencies install from a local wheel-house (`pip install --no-index --find-links vendor`). `pytest tests/ -q` and `python -m tests.smoke` both run with **every socket blocked**; the smoke run takes 1,000 flows through features → forecast → explanation → ledger → verification in ~9 s.